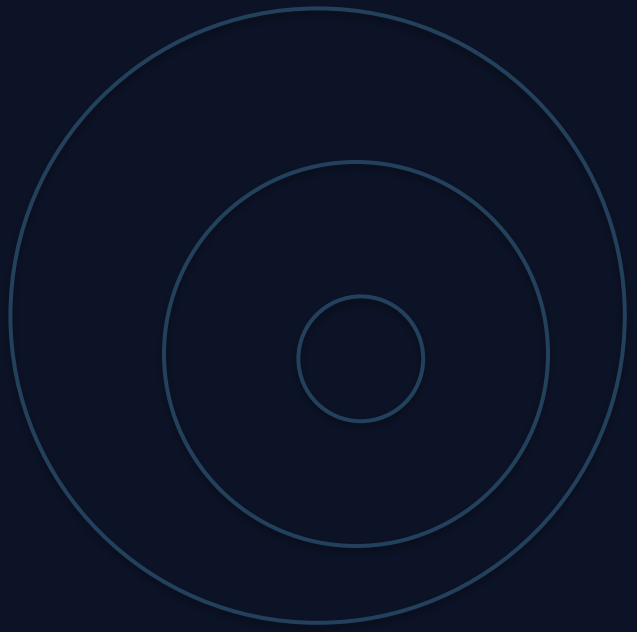
END-TO-END DELIVERY ENGINEERING

DevOps Capstone Project



From source code to containers, Kubernetes, cloud infrastructure, security gates and observability.

FLASK | DOCKER | KUBERNETES | TERRAFORM | DEVSECOPS

Project walkthrough • Evidence-led presentation • October 2026

What the project demonstrates

Application

A stateless Flask service with health, readiness and Prometheus metrics endpoints.

Delivery automation

GitHub Actions workflow for tests, security checks, image publishing and deployment.

Infrastructure & runtime

Terraform-managed AWS VPC, ECR and EKS; Helm/Kubernetes deployment patterns.

Security by design

Bandit, pip-audit, Gitleaks and Trivy are positioned as gates before release.

Operational readiness

Health checks, logs, Prometheus/Grafana monitoring, GitOps and troubleshooting exercises.

Core principle: make delivery repeatable, observable, and safer—not just “get the app running.”

End-to-end workflow

01 Source & tests

Flask app • pytest

02 Quality & security

Bandit • pip-audit • Gitleaks

03 Build & scan

Docker • Trivy

04 Publish image

Amazon ECR • commit SHA

05 Deploy

Helm • Kubernetes / EKS

06 Operate

Health • metrics • alerts • GitOps

Release policy: the security gate must pass before an image is promoted or deployed.

Application packaged as a container

Container runtime evidence

- Multi-stage Docker build
- Non-root runtime user
- Gunicorn serving Flask on port 5000
- Health check reports container health

```
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ docker run -d --name devops-app
-p 5000:5000 devops-app:1.0.0
0b499b940b2b5b74d9b8612115e184ee538885b80f87c50389468a3be825085c
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS                   PORTS
                                  NAMES
0b499b940b2b   devops-app:1.0.0   "gunicorn --bind 0.0…"   9 seconds ago   Up 7 seconds (healthy)   0.0.0.0:500
0->5000/tcp, [::]:5000->5000/tcp   devops-app
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ docker logs devops-app
[2026-10-16 04:14:28 +0000] [1] [INFO] Starting gunicorn 23.0.0
[2026-10-16 04:14:28 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-10-16 04:14:28 +0000] [1] [INFO] Using worker: sync
[2026-10-16 04:14:28 +0000] [7] [INFO] Booting worker with pid: 7
[2026-10-16 04:14:28 +0000] [8] [INFO] Booting worker with pid: 8
```

Validation command

```
curl http://localhost:5000/health
```

```
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ curl http://localhost:5000/healt
h
{"app":"devops-app","environment":"development","status":"healthy","version":"1.0.0"}
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$
```

Observed response: healthy • version 1.0.0 • development environment

Local Kubernetes deployment with Helm

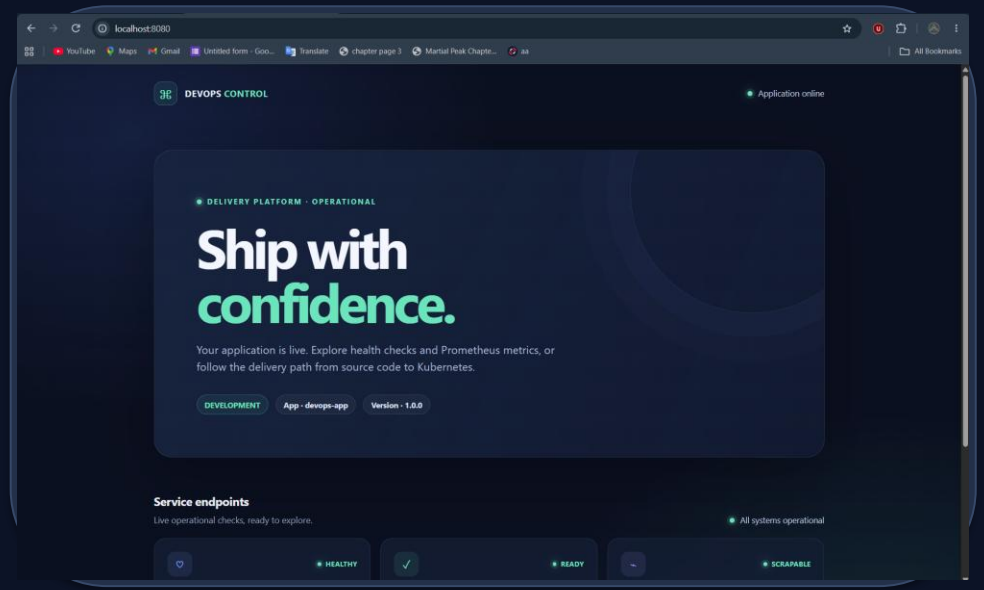
What was verified

- Minikube node reports Ready
- Image loaded into the local cluster
- Helm release reports deployed
- Pod and ClusterIP Service are present
- Application page is reachable via port-forward

```
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ kubectl config use-context minik
ube
Switched to context "minikube".
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ kubectl get nodes
NAME       STATUS   ROLES           AGE     VERSION
minikube   Ready    control-plane   7d10h   v1.37.0
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ minikube image load devops-app:1
.0.0
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ minikube image load devops-app:1
.0.0 .
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ minikube image ls | grep devops-
app
docker.io/library/devops-app:1.0.0
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$
```

```
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ kubectl get pods,services -n dev
ops-dev
NAME                              READY   STATUS    RESTARTS   AGE
pod/devops-app-794645f87-b4829    1/1     Running   0          18s

NAME                         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
service/devops-app-service   ClusterIP   10.109.121.241   <none>        80/TCP    18s
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ helm status devops-app -n devops
-dev
NAME: devops-app
LAST DEPLOYED: Fri Oct 16 05:02:02 2026
NAMESPACE: devops-dev
STATUS: deployed
REVISION: 1
```



The local Minikube run demonstrates the chart and app path without creating cloud resources.

Terraform provisions the AWS foundation

Network

VPC • public/private subnets • route tables • NAT

Container registry

ECR repository • image lifecycle policy

Kubernetes platform

EKS cluster • managed node group • IAM/OIDC

Validation evidence

```
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

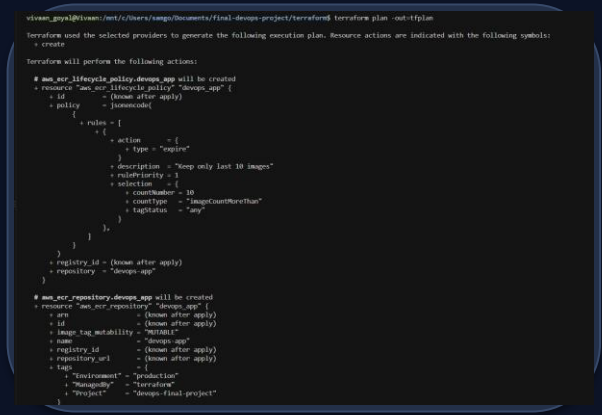
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
```

```
vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project/terraform$ terraform validate
Success! The configuration is valid.

vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project/terraform$
```

Terraform execution plan

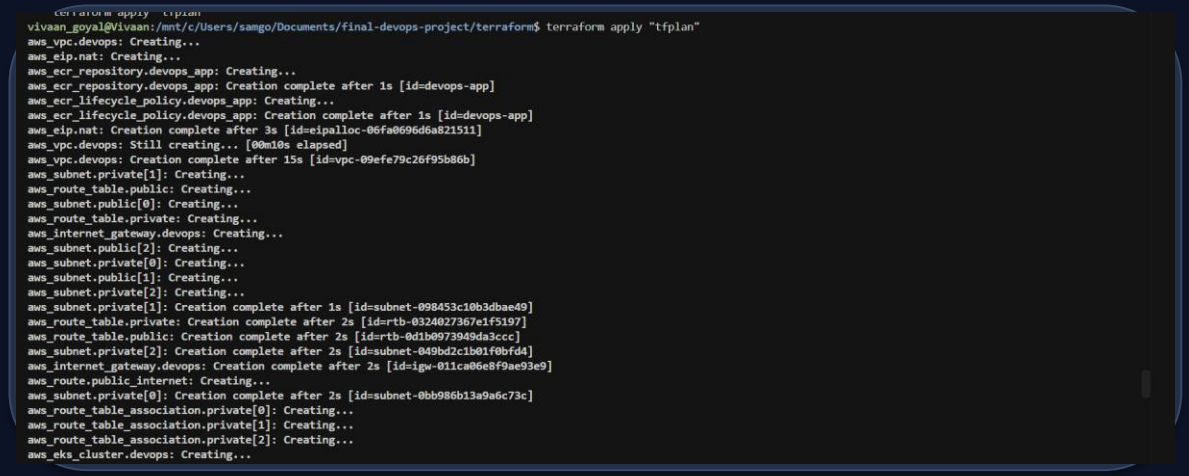


A valid plan is reviewable before apply; AWS resources are billable and should be destroyed after the lab.

Cloud provisioning and outputs

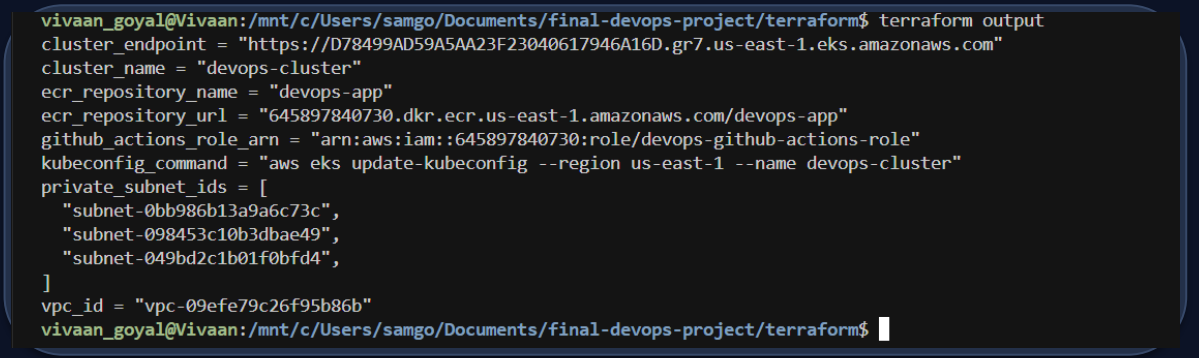
Provisioning evidence

Terraform created the ECR repository, VPC components, NAT gateway and route resources. EKS cluster creation takes longer than foundational networking.



Outputs captured

- EKS cluster name and endpoint
- ECR repository URI
- VPC and private subnet IDs
- GitHub Actions OIDC role ARN



Keep Terraform state and tfvars private; never commit credentials or sensitive state files.

CI/CD release path

Pull request

Tests + SAST + SCA + secret scan

→

Main branch

Build image + tag with commit SHA

→

Registry

Push image to Amazon ECR

→

Security gate

Trivy scan blocks critical/high findings

→

Deployment

Helm release to EKS

Release traceability

Commit-SHA tags connect a deployed image to source history.

Single deployment owner

Choose either direct CI deployment or Argo CD auto-sync for a release.

CI/CD pipeline: successful end-to-end run

GitHub Actions run #9 • main branch • commit 5f40e88 • Total duration 1m 45s



All jobs succeeded

Test, Bandit, pip-audit, Gitleaks, build/push, Trivy, security gate and EKS/Helm deployment.

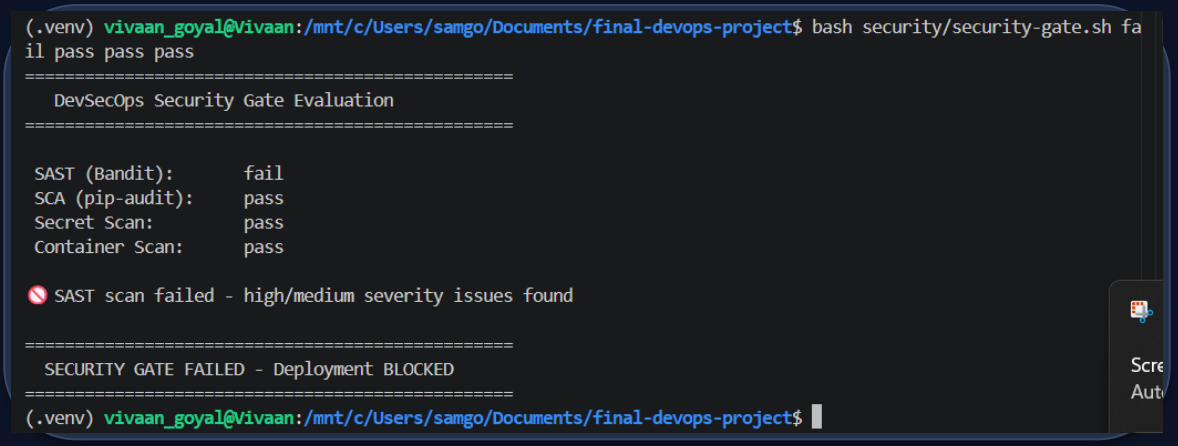
Secret scan

Gitleaks summary: no leaks detected.

Release outcome

The captured run shows a successful deployment workflow.

DevSecOps gate: fail closed



Observed local scan result

SAST (Bandit): FAIL
SCA (pip-audit): PASS
Secret scan: PASS
Container scan: PASS

Correct interpretation

The gate correctly blocks deployment when Bandit reports medium/high findings. Fix the findings and rerun the gate before claiming a green release.

A failed security gate is useful evidence: it demonstrates enforcement, not a successful production release.

Operate, observe, and reconcile

Application signals

/health • /ready • /metrics
Gunicorn/container logs

Monitoring stack

Prometheus Operator
Grafana dashboard
Alertmanager rules

GitOps option

Argo CD syncs Helm from Git
Kustomize is an alternative

Operational checks

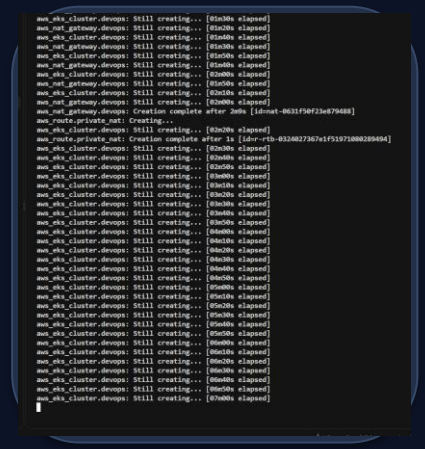
1. Check pod readiness and events.
2. Inspect application logs and health endpoints.
3. Confirm Prometheus target discovery and dashboard data.
4. Use one deployment owner to avoid CI and Argo CD overwriting each other.

Troubleshooting is part of the workflow

```
(.venv) vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ kubectl rollout status deploymen
t/devops-app -n devops-dev
Waiting for deployment "devops-app" rollout to finish: 0 of 1 updated replicas are available...
```

Local rollout checkpoint

One captured command was still waiting for an updated replica. Investigate with describe, logs, events and image-pull status rather than assuming success.



AWS cluster checkpoint

The screenshot shows EKS creation still in progress at that moment. Verify final cluster/node status and complete a deployment before reporting the cloud path as finished.

Troubleshooting record: symptom → evidence → root cause → fix → verification.

Repository structure and reproducibility

application/	Flask app and unit tests
docker/	Dockerfile and container build
kubernetes/ • helm/	Standalone manifests and reusable Helm chart
terraform/	AWS VPC, ECR, EKS and GitHub OIDC
.github/workflows/ • security/	CI/CD automation and security gate
monitoring/ • gitops/	Prometheus/Grafana resources, Argo CD and Kustomize

Guided local path: bash ./run-local.sh • Deployment details: DEPLOY.md • Failure labs: TROUBLESHOOTING.md

Key takeaways

Automate repeatable steps

Tests, image builds, security scans and deployments should be consistent.

Gate before release

Security checks should fail closed and findings must be resolved.

Infrastructure is code

Review Terraform plans, restrict access, protect state and manage cloud costs.

Verify what you claim

Use health checks, rollout status, logs, events and monitoring evidence.

Outcome: a practical delivery platform that connects application development, infrastructure, security, deployment and operations.

Evidence index

01 Docker container running + Gunicorn startup logs

02 Health endpoint response from localhost:5000

03 Minikube node/image and Helm release status

04 Application landing page served through Kubernetes

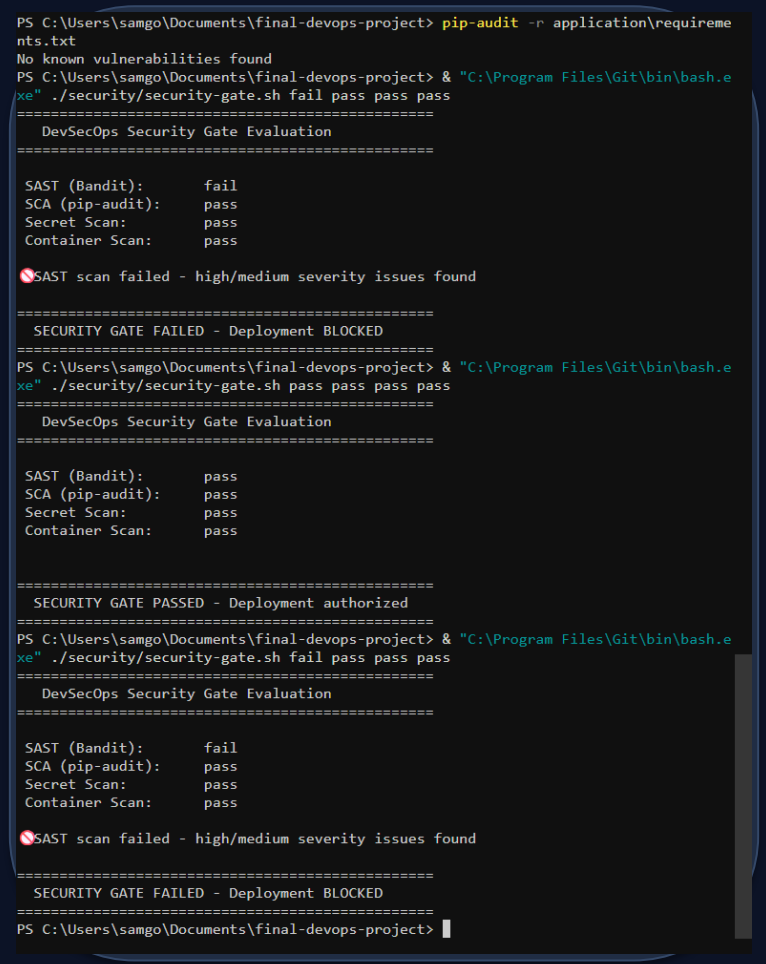
05 Terraform init, validate, plan, apply and outputs

06 Bandit security gate failure with other checks passing

Security note: IAM access-key and organization-policy screenshots were omitted from this deck.

Security gate: block on findings, pass when clean

Local gate script exercised with both failing and passing inputs



FAIL-CLOSED BEHAVIOR

Bandit SAST failure blocks deployment even when SCA, secret scanning and container scanning pass.

AUTHORIZED PATH

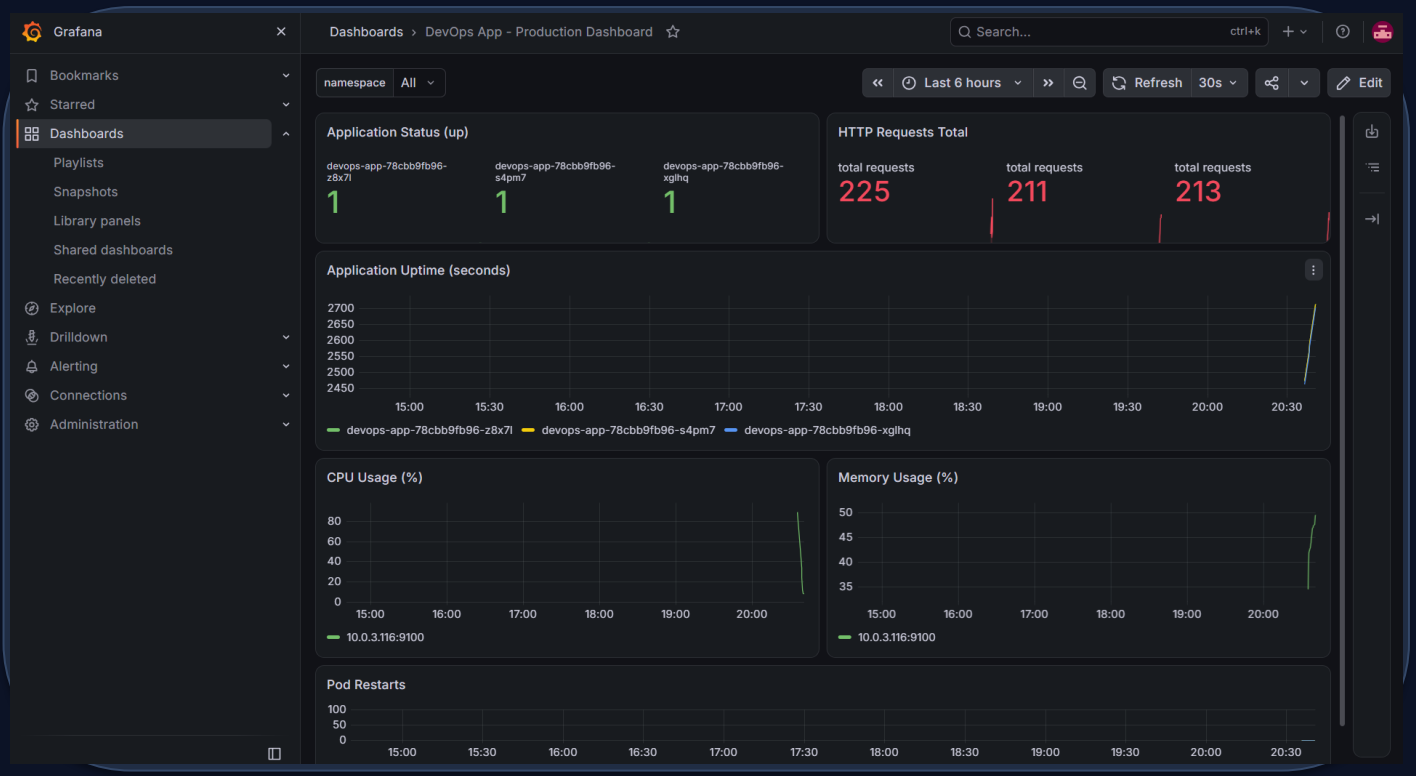
When all four checks pass, the script reports “SECURITY GATE PASSED — Deployment authorized.”

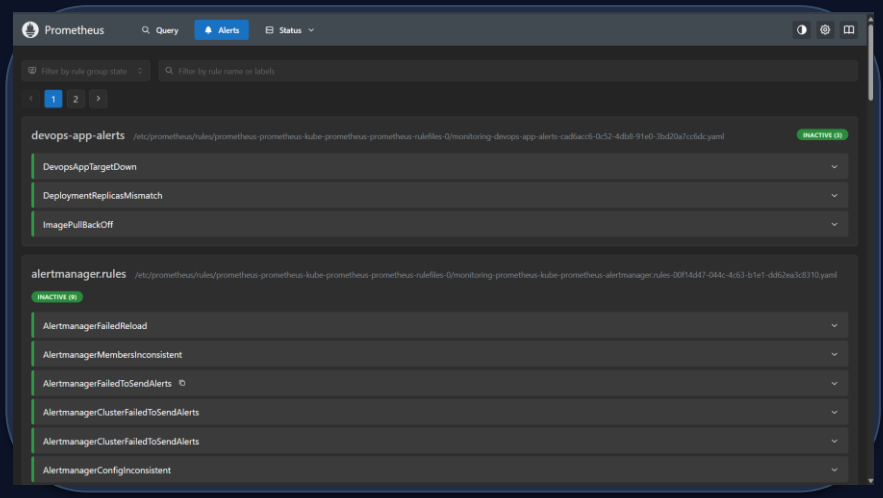
WHY IT MATTERS

A visible policy decision turns scanner output into an explicit release control—not just a report.

Monitoring: application signals and alert rules

Grafana provides a runtime view; Prometheus exposes configured alert rules.





GRAFANA

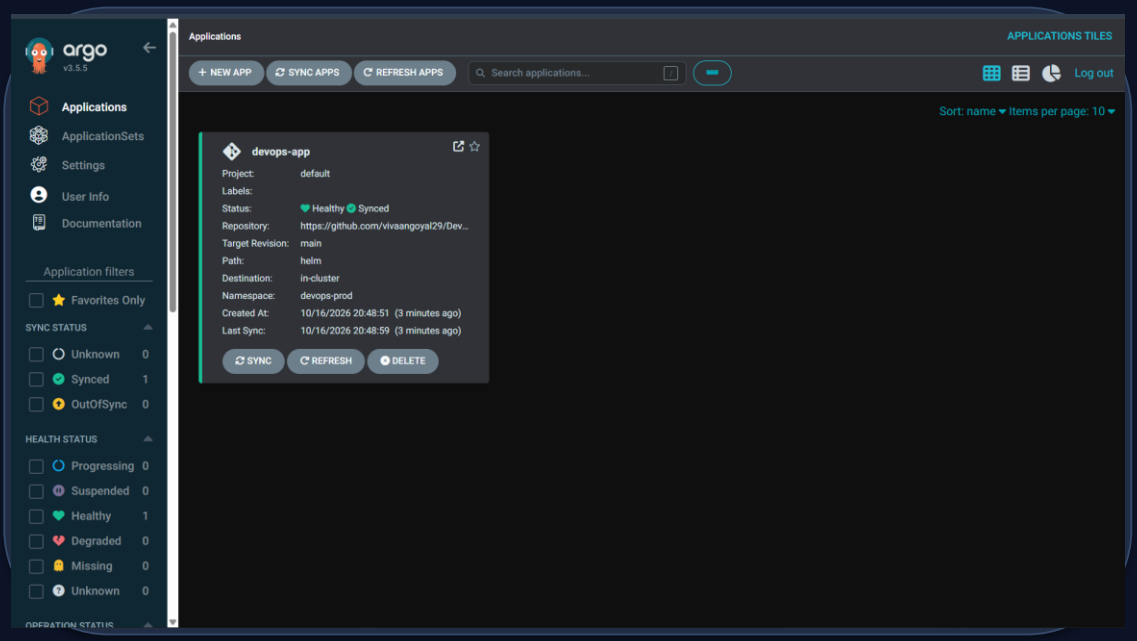
App up, HTTP request totals, uptime, CPU, memory and pod restarts.

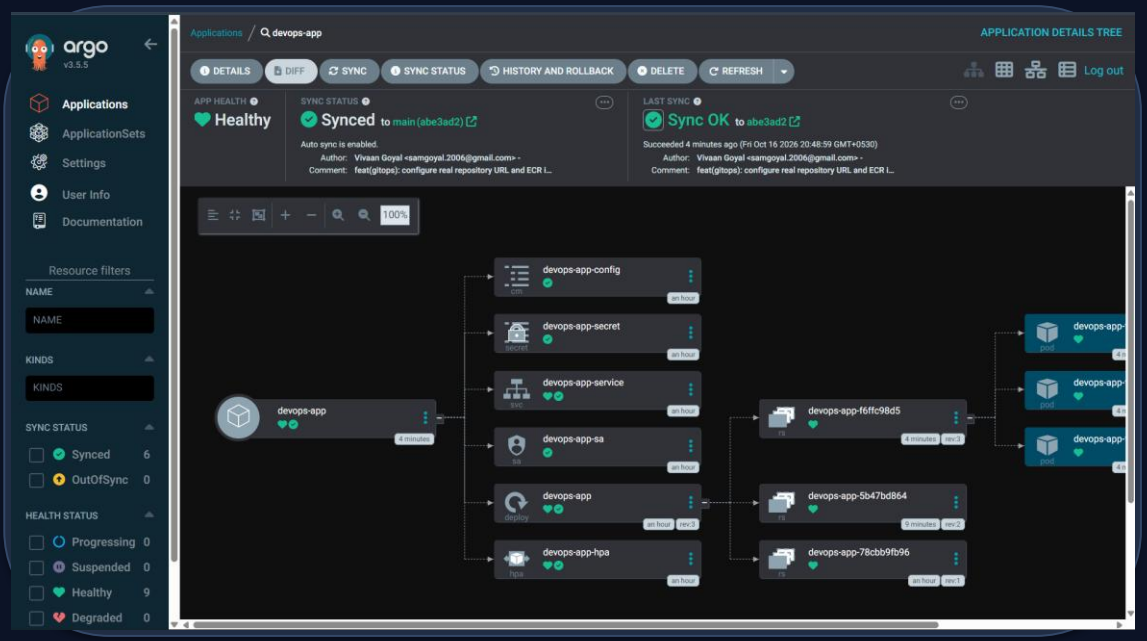
PROMETHEUS

Rules include target-down, replica mismatch and image-pull backoff.

Argo CD: application healthy and synced

The deployment is reconciled from the main branch into the devops-prod namespace.





HEALTH

Argo CD reports the application as Healthy.

SYNC

Sync status is Synced to main; last sync is successful.

RESOURCES

Tree view shows config, secret, service, deployment, HPA, ReplicaSets and pods.

Kubernetes: diagnose a liveness-probe failure

Event output records an HTTP 404 from the liveness probe; a later pod listing shows a running, ready pod.

```
PS C:\Users\samgo\Documents\final-devops-project> kubectl get pods -n devpos-dev
No resources found in devpos-dev namespace.
PS C:\Users\samgo\Documents\final-devops-project> kubectl get events -n devops-dev -
-sort-by=.metadata.creationTimestamp | Select-Object -Last 10
LAST SEEN   TYPE      REASON              OBJECT                                  MESSAGE
3m6s        Normal    Scheduled           pod/devops-app-67fcbf7549-b4sxd         Success
fully assigned devops-dev/devops-app-67fcbf7549-b4sxd to ip-10-0-2-95.ec2.internal
5s          Normal    Pulled              pod/devops-app-67fcbf7549-b4sxd         Contain
er image "645897840730.dkr.ecr.us-east-1.amazonaws.com/devops-app:latest" already pr
esent on machine and can be accessed by the pod
5s          Normal    Created             pod/devops-app-67fcbf7549-b4sxd         Contain
er created
5s          Normal    Started             pod/devops-app-67fcbf7549-b4sxd         Contain
er started
3m6s        Normal    SuccessfulCreate    replicaset/devops-app-67fcbf7549        Created
 pod: devops-app-67fcbf7549-b4sxd
3m6s        Normal    ScalingReplicaSet   deployment/devops-app                   Scaled
up replica set devops-app-67fcbf7549 from 0 to 1
6s          Warning   Unhealthy           pod/devops-app-67fcbf7549-b4sxd         Livenes
s probe failed: HTTP probe failed with statuscode: 404
6s          Normal    Killing             pod/devops-app-67fcbf7549-b4sxd         Contain
er devops-app failed liveness probe, will be restarted
PS C:\Users\samgo\Documents\final-devops-project>
```

```
vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$ kubectl get p
ods -n devops-dev
NAME                          READY   STATUS    RESTARTS   AGE
devops-app-794645f87-b4829    1/1     Running   0          10h
vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project$
```

DIAGNOSTIC SIGNAL

The event stream captures the failed probe and container restart.

RUNTIME CHECK

A separate later `kubectl get pods` capture shows 1/1 Ready and Running in devops-dev.

Evidence is presented as investigation + subsequent runtime state; the screenshots do not prove the same pod recovered.

Terraform plan: review infrastructure changes before apply

The captured plan includes an ECR repository and lifecycle policy to retain only the latest 10 images.

```
vivaan_goyal@Vivaan:/mnt/c/Users/samgo/Documents/final-devops-project/terraform$ terraform plan -out=tfplan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_ecr_lifecycle_policy.devops_app will be created
  + resource "aws_ecr_lifecycle_policy" "devops_app" {
      + id          = (known after apply)
      + policy      = jsonencode(
            {
              + rules = [
                  + {
                      + action       = {
                          + type = "expire"
                        }
                      + description  = "Keep only last 10 images"
                      + rulePriority = 1
                      + selection    = {
                          + countNumber = 10
                          + countType   = "imageCountMoreThan"
                          + tagStatus   = "any"
                        }
                    },
                ]
            }
        )
      + registry_id = (known after apply)
      + repository  = "devops-app"
    }

  # aws_ecr_repository.devops_app will be created
  + resource "aws_ecr_repository" "devops_app" {
      + arn                  = (known after apply)
      + id                   = (known after apply)
      + image_tag_mutability = "MUTABLE"
      + name                 = "devops-app"
      + registry_id          = (known after apply)
      + repository_url       = (known after apply)
      + tags                 = {
          + "Environment" = "production"
          + "ManagedBy"   = "terraform"
          + "Project"     = "devops-final-project"
```

PLAN FIRST

Terraform previews proposed resource changes before they are applied.

ECR REPOSITORY

Repository name: devops-app; tags identify environment and project.

IMAGE RETENTION

Lifecycle policy expires images beyond the configured count of 10.